

Module 4

Pig, Hive, HBase

Module 4 Contents

Pig : Introduction to PIG, Execution Modes of Pig, Comparison of Pig with Databases, Grunt, Pig Latin, User Defined Functions, Data Processing operators.

Hive : Hive Shell, Hive Services, Hive Metastore, Comparison with Traditional Databases, HiveQL, Tables, Querying Data and User Defined Functions.

Hbase : HBasics, Concepts, Clients, Example, Hbase Versus RDBMS.

PIG - Introduction

- Pig is a high level data flow **platform for analyzing large datasets** stored in hadoop clusters.
- Pig **simplifies the process of writing complex MapReduce programs** in Java, by offering a more user-friendly scripting language (**Pig Latin**).
- The **Pig scripts get internally converted to Mapreduce jobs** and get executed on data stored in hadoop.
- Pig can handle any type of data, i.e., structured, semi-structured or unstructured and stores the corresponding results into Hadoop Data File System.
- Every tasks (MapReduce) which can be achieved using Java can also be achieved using PIG latin.

Why was Pig Created?

- In a MapReduce framework, programs need to be translated into a series of Map and Reduce stages.
- Data Analysts are not familiar with this, to bridge this gap an abstraction called Pig was built on top of Hadoop
- Apache Pig reduces the time of development using the multi-query approach.
- Pig is beneficial for programmers who are not from Java background.

Features

- **Ease of programming:** Writing complex java programs for map-reduce is quite tough for non-programmers, pig makes this process easy (the queries are converted to MapReduce internally).
- **Optimization opportunities:** By encoding tasks in a certain way, the system is able to automatically optimize their execution, allowing the user to concentrate on semantics rather than efficiency.
- **Extensibility:** A user-defined function is written in which the user can write their logic to execute over the data set.
- **Flexible:** It can easily handle structured as well as unstructured data.
- **In-built operators:** It contains various type of operators such as sort, filter and joins.

Anatomy of PIG

Contains

1. Data flow language PIG-Latin
2. Grunt - Interactive shell where we can type Pig Latin Statement
3. Pig Interpreter and execution engine.

Advantages of Apache Pig

- **Less code** - The Pig consumes less line of code to perform any operation.
- **Reusability** - The Pig code is flexible enough to reuse again.
- **Nested data types** - The Pig provides a useful concept of nested data types like tuple.

Apache Pig Execution Modes

- We can run Apache Pig in two modes, namely, Local Mode and MapReduce mode.
- **Local Mode**
 - In this mode, all the files are installed and run from your local host and local file system.
 - There is no need of Hadoop or HDFS.
 - This mode is generally used for testing purpose.
- **MapReduce Mode**
 - MapReduce mode is where we load or process the data that exists in the Hadoop File System (HDFS) using Apache Pig.
 - its also known as hadoop mode.
 - In this mode, whenever we execute the Pig Latin statements to process the data, a MapReduce job is invoked in the back-end to perform a particular operation on the data that exists in the HDFS.

Apache Pig Execution Mechanisms

- **Interactive Mode (Grunt shell)**

- We can run Apache Pig in interactive mode using the Grunt shell.
- In this shell, we can enter the Pig Latin statements and get the output (using Dump operator).

- **Batch Mode (Script)**

- We can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with .pig extension.

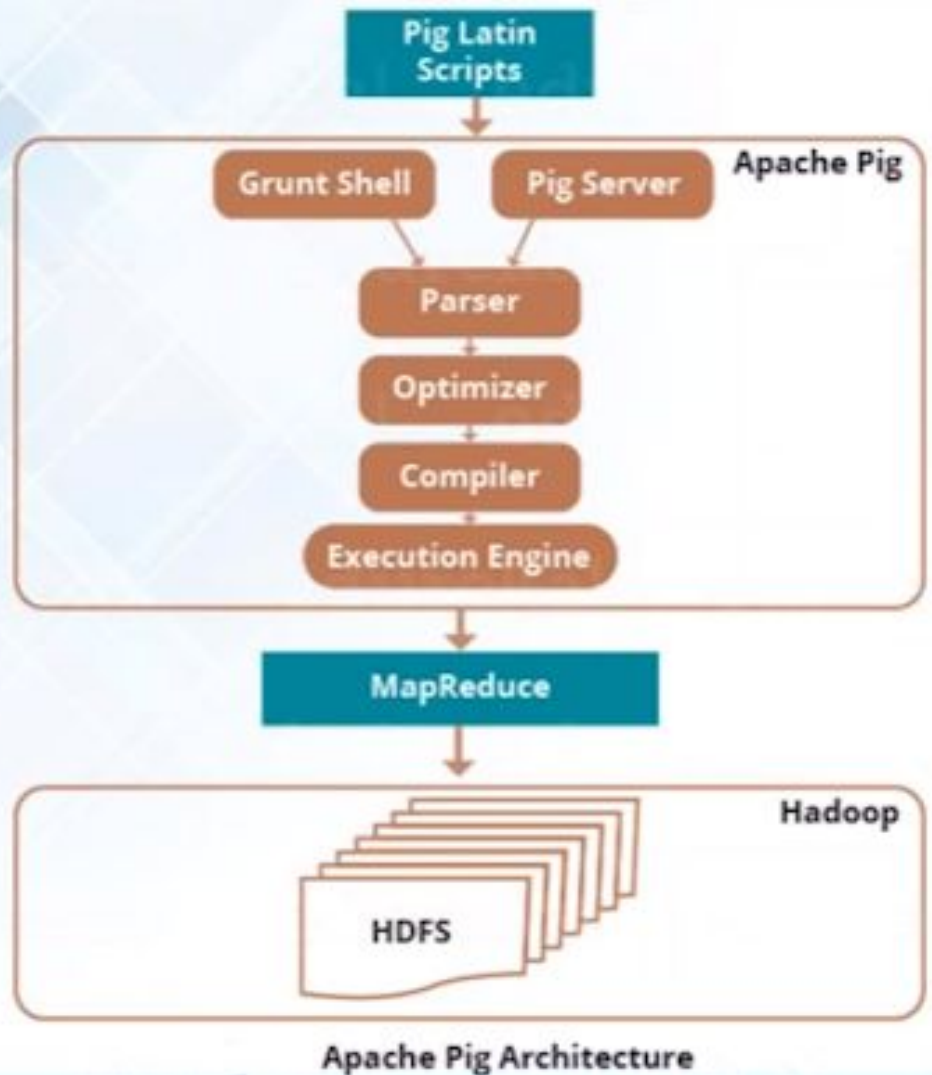
- **Embedded Mode (UDF)**

- Apache Pig provides the provision of defining our own functions (User Defined Functions) in programming languages such as Java, and using them in our script.

Comparison with Database

Pig	SQL
Pig Latin is a procedural language.	SQL is a declarative language.
In Apache Pig, schema is optional. We can store data without designing a schema (values are stored as \$01, \$02 etc.)	Schema is mandatory in SQL.
The data model in Apache Pig is nested relational .	The data model used in SQL is flat relational .
Apache Pig provides limited opportunity for Query optimization .	There is more opportunity for query optimization in SQL.

Apache Pig - Architecture



Pig Latin

- The language used to analyze data in Hadoop using Pig is known as Pig Latin.
- It is a high level data processing language which provides a rich set of data types and operators to perform various operations on the data.
- Series of operations or transformations are applied to the input data to produce results.

Pig script

- To perform a particular task with Pig platform programmers need to write a Pig script using the Pig Latin language, and execute them using any of the execution mechanisms (Grunt Shell, UDFs, Embedded).
- After execution, these scripts will go through a series of transformations applied by the Pig Framework, to produce the desired output.

Grunt Shell

- Interactive shell for running pig commands

Parser

- Pig Scripts are handled by the Parser, It **checks the syntax** of the script.
- The output of the parser will be a **DAG** (directed acyclic graph), which represents the Pig Latin statements and logical operators.
- In the DAG, the logical operators of the script are represented as the nodes and the data flows are represented as edges.

Optimizer

- The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as **projection** and **pushdown**.

Compiler

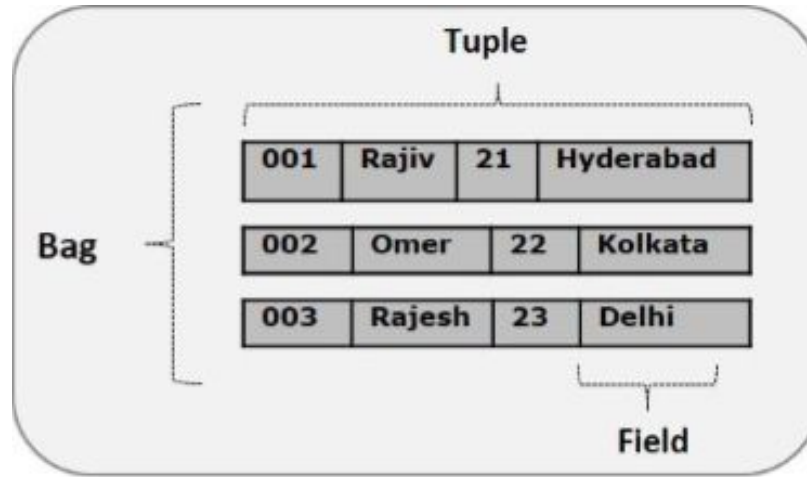
- The compiler compiles the optimized logical plan into a series of MapReduce jobs.

Execution engine

- Finally the MapReduce jobs are submitted to Hadoop in a sorted order.
- These MapReduce jobs are executed on Hadoop producing the desired results.

Pig Latin Data Model

- The data model of Pig Latin is fully nested and it allows complex non-atomic data types such as **map** and **tuple**.
- Given below is the diagrammatic representation of Pig Latin's data model



Atom

- Any single value in Pig Latin, irrespective of their data type is known as an atom.
- It is stored as string and can be used as string and number.
- int, long, float, double, chararray, and bytearray are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.
- Example – 'raja' or '30'

Tuple

- A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.
- Example – (Raja, 30)

Bag

- A bag is an unordered set of tuples, each tuple can have any number of fields (flexible schema).
- A bag is represented by '{}'.
 - Example – {(Raja, 30), (Mohammad, 45)}

Map

- A map is a set of key-value pairs.
- The key needs to be of type **chararray** and should be unique.
- The value might be of any type.
- It is represented by '[']. Example – [name#Raja, age#30]

Relation

- A relation is a bag of tuples.
- The relations in Pig Latin are unordered ,there is no guarantee that tuples are processed in any particular order).

Grunt

Grunt is Pig's interactive shell. It enables users to enter Pig Latin interactively and provides a shell for users to interact with HDFS.

To enter Grunt, invoke Pig with no script or command to run.

```
pig -x local
```

will result in the prompt:

```
grunt>
```

This gives you a Grunt shell to interact with your local filesystem.

- Grunt provides command-line history and editing, as well as Tab completion.
- You can enter Pig Latin directly into Grunt.
- Unix shells and will operate in ways that are familiar: chgrp, chmod, chown, cp, du, ls, mkdir, mv, rm, and stat.
- **cat filename**
 - Print the contents of a file to stdout.
 - You can apply this command to a directory and it will apply itself in turn to each file in the directory.
- **copyFromLocal localfile hdfsfiler**
 - Copy a file from your local disk to HDFS. This is done serially, not in parallel.

- **copyToLocal hdfsfile localfile**

- Copy a file from HDFS to your local disk. This is done serially, not in parallel.

- **rmr filename**

- Remove files recursively. This is equivalent to `rm -r` in Unix.
- Pig also used to control the program.

kill jobid

exec [[-param param_name = param_value]] [[-param_file filename]]

script

run [[-param param_name = param_value]] [[-param_file filename]] script

Pig Latin

Structure

- A Pig Latin program consists of a collection of statements.
- A statement can be thought of as an operation or a command.
- For example, a GROUP operation is a type of statement:
 - `grouped_records = GROUP records BY year;`
- The command to list the files in a Hadoop filesystem is another example of a statement.
 - `ls /`
- Statements are usually terminated with a semicolon.
- Grunt do not need the terminating semicolon

Table 16-1. Pig Latin relational operators

Category	Operator	Description
Loading and storing	LOAD	Loads data from the filesystem or other storage into a relation
	STORE	Saves a relation to the filesystem or other storage
	DUMP (\d)	Prints a relation to the console
Filtering	FILTER	Removes unwanted rows from a relation
	DISTINCT	Removes duplicate rows from a relation
	FOREACH...GENERATE	Adds or removes fields to or from a relation
	MAPREDUCE	Runs a MapReduce job using a relation as input
	STREAM	Transforms a relation using an external program
	SAMPLE	Selects a random sample of a relation
	ASSERT	Ensures a condition is true for all rows in a relation; otherwise, fails
Grouping and joining	JOIN	Joins two or more relations
	COGROUP	Groups the data in two or more relations
	GROUP	Groups the data in a single relation
	CROSS	Creates the cross product of two or more relations
	CUBE	Creates aggregations for all combinations of specified columns in a relation
Sorting	ORDER	Sorts a relation by one or more fields
	RANK	Assign a rank to each tuple in a relation, optionally sorting by fields first
	LIMIT	Limits the size of a relation to a maximum number of tuples
Combining and splitting	UNION	Combines two or more relations into one
	SPLIT	Splits a relation into two or more relations

Comments

1. Double hyphens are used for single-line comments.

```
-- My program
```

```
DUMP A;
```

```
-- What's in A?
```

2. C-style comments are more flexible since they delimit the beginning and end of the comment block with `/*` and `*/` markers.

Table 16-4. Pig Latin commands

Category	Command	Description
Hadoop filesystem	cat	Prints the contents of one or more files
	cd	Changes the current directory
	copyFromLocal	Copies a local file or directory to a Hadoop filesystem
	copyToLocal	Copies a file or directory on a Hadoop filesystem to the local filesystem
	cp	Copies a file or directory to another directory
	fs	Accesses Hadoop's filesystem shell
	ls	Lists files
	mkdir	Creates a new directory
	mv	Moves a file or directory to another directory
	pwd	Prints the path of the current working directory
	rm	Deletes a file or directory
	rmf	Forcibly deletes a file or directory (does not fail if the file or directory does not exist)
Hadoop MapReduce	kill	Kills a MapReduce job

Category	Command	Description
Utility	clear	Clears the screen in Grunt
	exec	Runs a script in a new Grunt shell in batch mode
	help	Shows the available commands and options
	history	Prints the query statements run in the current Grunt session
	quit (\q)	Exits the interpreter
	run	Runs a script within the existing Grunt shell
	set	Sets Pig options and MapReduce job properties
	sh	Runs a shell command from within Grunt

- The `filesystem` commands can operate on files or directories in any Hadoop filesystem, and they are very similar to the `hadoop fs` commands
- You can access all of the Hadoop filesystem shell commands using Pig's `fs` command.
- Example
 - `fs -ls /`

- REGISTER, DEFINE, and IMPORT—that make it possible to incorporate macros and user-defined functions into Pig scripts

Table 16-3. Pig Latin macro and UDF statements

Statement	Description
REGISTER	Registers a JAR file with the Pig runtime
DEFINE	Creates an alias for a macro, UDF, streaming script, or command specification
IMPORT	Imports macros defined in a separate file into a script

exec command in PIG

- Syntax
 - `exec [-param param_name = param_value] [-param_file file_name] [script]`
- Use the **exec** command to run a Pig script with **no interaction** between the script and the Grunt shell (batch mode).
- The files produced as the output of the script and stored on the system are visible after the script is run.
- Here the entire script is parsed before execution starts.

run command in PIG

- Use the run command to run a Pig script that **can interact** with the Grunt shell (interactive mode).
- All commands from the script are visible in the command history.
- Both the exec and run commands are useful for debugging because you can modify a Pig script in an editor and then rerun the script in the Grunt shell without leaving the shell.

Control Flow

- By design, Pig Latin lacks native control flow statements.
- The recommended approach for writing programs that have conditional logic or loop constructs is to embed Pig Latin in another language, such as Python, JavaScript, or Java, and manage the control flow from there.

Pig Latin- Expression

Category	Expressions	Description	Examples
Constant	Literal	Constant value (see also the “Literal example” column in Table 16-6)	1.0, 'a'
Field (by position)	$\$n$	Field in position n (zero-based)	$\$0$
Field (by name)	f	Field named f	year
Field (disambiguate)	$r::f$	Field named f from relation r after grouping or joining	$A::year$
Projection	$c.\$n, c.f$	Field in container c (relation, bag, or tuple) by position, by name	$records.\$0, records.year$
Map lookup	$m\#k$	Value associated with key k in map m	$items\#'Coat'$
Cast	$(t) f$	Cast of field f to type t	$(int) year$

Category	Expressions	Description	Examples
Arithmetic	$x + y, x - y$	Addition, subtraction	$\$1 + \$2, \$1 - \2
	$x * y, x / y$	Multiplication, division	$\$1 * \$2, \$1 / \2
	$x \% y$	Modulo, the remainder of x divided by y	$\$1 \% \2
	$+x, -x$	Unary positive, negation	$+1, -1$
Conditional	$x ? y : z$	Bincond/ternary; y if x evaluates to true, z otherwise	$\text{quality} == 0 ? 0 : 1$
	CASE	Multi-case conditional	CASE q WHEN 0 THEN 'good' ELSE 'bad' END

Category	Expressions	Description	Examples
Comparison	<code>x == y, x != y</code>	Equals, does not equal	<code>quality == 0, temperature != 9999</code>
	<code>x > y, x < y</code>	Greater than, less than	<code>quality > 0, quality < 10</code>
	<code>x >= y, x <= y</code>	Greater than or equal to, less than or equal to	<code>quality >= 1, quality <= 9</code>
	<code>x matches y</code>	Pattern matching with regular expression	<code>quality matches '[01459]'</code>
	<code>x is null</code>	Is null	<code>temperature is null</code>
	<code>x is not null</code>	Is not null	<code>temperature is not null</code>

Category	Expressions	Description	Examples
Boolean	$x \text{ OR } y$	Logical OR	$q == 0 \text{ OR } q == 1$
	$x \text{ AND } y$	Logical AND	$q == 0 \text{ AND } r == 0$
	$\text{NOT } x$	Logical negation	$\text{NOT } q \text{ matches '}[01459]'$
	$\text{IN } x$	Set membership	$q \text{ IN } (0, 1, 4, 5, 9)$
Functional	$fn(f_1, f_2, \dots)$	Invocation of function fn on fields f_1, f_2 , etc.	<code>isGood(quality)</code>
Flatten	$\text{FLATTEN}(f)$	Removal of a level of nesting from bags and tuples	<code>FLATTEN(group)</code>

Pig Latin- Data Types

Category	Type	Description	Literal example
Boolean	boolean	True/false value	true
Numeric	int	32-bit signed integer	1
	long	64-bit signed integer	1L
	float	32-bit floating-point number	1.0F
	double	64-bit floating-point number	1.0
	biginteger	Arbitrary-precision integer	'10000000000'
	bigdecimal	Arbitrary-precision signed decimal number	'0.1100010000000000000000000001'
Text	chararray	Character array in UTF-16 format	'a'
Binary	bytearray	Byte array	Not supported
Temporal	datetime	Date and time with time zone	Not supported, use ToDate built-in function
Complex	tuple	Sequence of fields of any type	(1, 'pomegranate')
	bag	Unordered collection of tuples, possibly with duplicates	{(1, 'pomegranate'), (2)}
	map	Set of key-value pairs; keys must be character arrays, but values may be any type	['a' #'pomegranate']

Schemas

- A relation in Pig may have an associated schema, which gives the fields in the relation names and types.
- An **AS clause** in a LOAD statement is used to attach a schema to a relation:
 - `records = LOAD 'input/ncdc/micro-tab/sample.txt' AS (year:int, temperature:int, quality:int);`
 - `DESCRIBE records;`
 - `records: {year: int,temperature: int,quality: int}`

- Define and initialize a variable
 - `DEFINE myVariable 'Hello, World!';`
- Use the variable in subsequent operations
 - `myRelation = LOAD 'input.txt' USING PigStorage(',') AS (col1: chararray, col2: chararray);`
 - `myFilteredRelation = FILTER myRelation BY col1 == $myVariable;`
- Store the result
 - `STORE myFilteredRelation INTO 'output.txt' USING PigStorage(',');`

- In this example, `myVariable` is initialized with the value `'Hello, World!'`.
- `variable myRelation` is used to load dataset from `input.txt` file as a relation
- Filtering the relation `myRelation` based on the `myVariable's` value.
- The results are stored in the `myFilteredRelation` relation and written to the `output.txt` file.

pig_final.pig

```
student = LOAD 'hdfs://localhost:9000/pig_data/details.txt' USING PigStorage(',')  
  as (id:int, firstname:chararray, lastname:chararray, age:int,phone:chararray,  
city:chararray);
```

```
student_order = ORDER student BY age DESC;
```

```
student_filtered= FILTER student BY age > 22;
```

```
STORE student_filtered INTO 'hdfs://localhost:9000/pig_data/sorted_data.txt'  
USING PigStorage(',');
```

```
dump student_filtered;
```


Pig Script in Batch mode

Step 1

Write all the required Pig Latin statements in a single file and save it as **.pig** file.

Step 2

Execute the Apache Pig script, can execute the Pig script from the Linux shell as shown below.

Local mode : \$ pig -x local sample_script.pig

MapReduce mode : \$ pig -x mapreduce sample_script.pig

Can execute it from the Grunt shell using the **exec** command as shown below.

```
grunt> exec /sample_script.pig
```

Executing a Pig Script from HDFS

We can also execute a Pig script that resides in the HDFS. Suppose there is a Pig script with the name `Sample_script.pig` in the HDFS directory named `/pig_data/`. We can execute it as shown below.

```
$ pig -x mapreduce hdfs://localhost:9000/pig_data/Sample_script.pig
```

- In some case, we have specified only the names of the fields in the schema: year, temperature, and quality. The types default to bytearray, the most general type, representing a binary string.
- You don't need to specify types for every field; you can leave some to default to bytearray,
- `records = LOAD 'input/ncdc/micro-tab/sample.txt';`
`grunt> DESCRIBE records;` Schema for records
unknown. Error

Functions

Eval function

A function that takes one or more expressions and returns another expression.

An example of a built-in eval function is MAX, which returns the maximum value of the entries in a bag.

aggregate functions-algebraic

Filter function

A special type of eval function that returns a logical Boolean result. As the name suggests, filter functions are used in the FILTER operator to remove unwanted rows.

An example of a built-in filter function is IsEmpty, which tests whether a bag or a map contains any items.

Load and Store

- **Load function**
 - A function that specifies how to load data into a relation from external storage.
- **Store function**
 - A function that specifies how to save the contents of a relation to external storage.
- Often, load and store functions are implemented by the same type.

Table 16-7. A selection of Pig's built-in functions

Category	Function	Description
Eval	AVG	Calculates the average (mean) value of entries in a bag.
	CONCAT	Concatenates byte arrays or character arrays together.
	COUNT	Calculates the number of non-null entries in a bag.
	COUNT_STAR	Calculates the number of entries in a bag, including those that are null.
	DIFF	Calculates the set difference of two bags. If the two arguments are not bags, returns a bag containing both if they are equal; otherwise, returns an empty bag.
	MAX	Calculates the maximum value of entries in a bag.
	MIN	Calculates the minimum value of entries in a bag.
	SIZE	Calculates the size of a type. The size of numeric types is always 1; for character arrays, it is the number of characters; for byte arrays, the number of bytes; and for containers (tuple, bag, map), it is the number of entries.
	SUM	Calculates the sum of the values of entries in a bag.
	TOBAG	Converts one or more expressions to individual tuples, which are then put in a bag. A synonym for ().
	TOKENIZE	Tokenizes a character array into a bag of its constituent words.
	TOMAP	Converts an even number of expressions to a map of key-value pairs. A synonym for [].

User Defined Function (UDF)

- In addition to the built-in functions, Apache Pig provides extensive support for User Defined Functions (UDF's).
- The UDF support is provided in six programming languages, namely, Java, Jython, Python, JavaScript, Ruby and Groovy.
- Among all the languages, Pig provides the most extensive support for Java functions and limited support is provided in all the remaining languages.

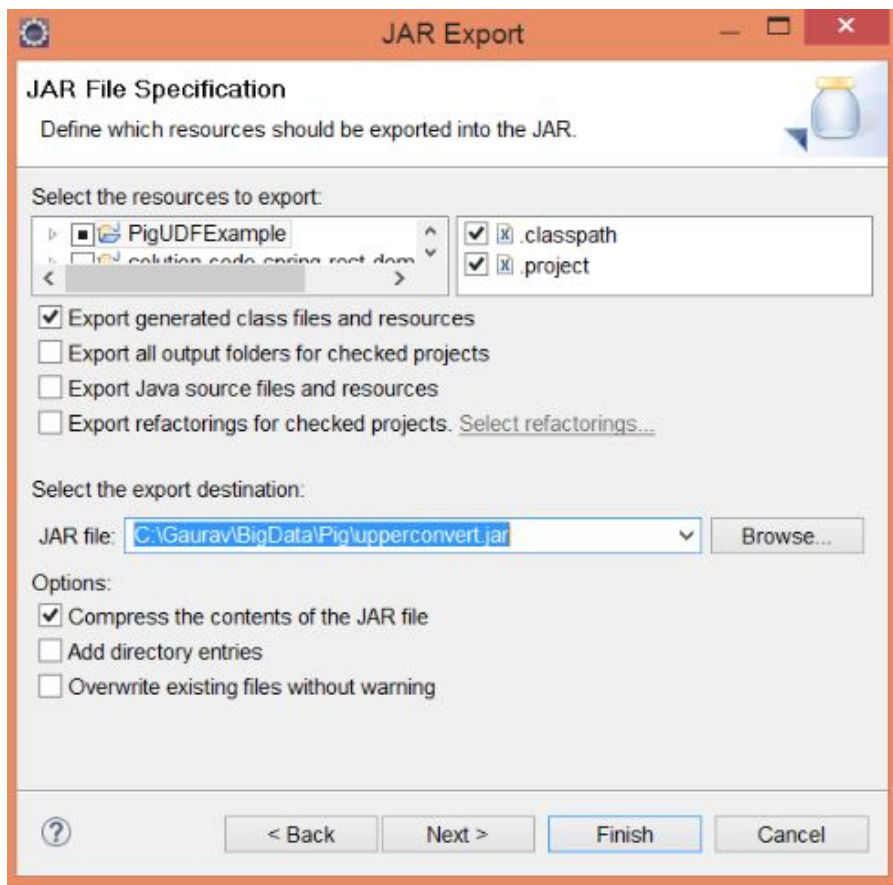
Types of UDF's in Java

- We can create and use the following **three types** of functions –
- **Filter Functions**
 - The filter functions are used as conditions in **FILTER** statements.
 - These functions accept a Pig value as input and return a Boolean value.
- **Eval Functions**
 - The Eval functions are used in **FOREACH-GENERATE** statements.
 - These functions accept a Pig value as input and return a Pig result.
- **Algebraic Functions**
 - The Algebraic functions act on inner bags in a **FOREACH-GENERATE** statement.
 - These functions are used to perform full MapReduce operations on an inner bag.

- Example of a simple **EVAL** Function to convert the provided string to uppercase.
- All UDFs must extend "org.apache.pig.EvalFunc"
- All functions must override the "exec" method

```
package com.hadoop;
import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
public class TestUpper extends EvalFunc<String>
{
    public String exec(Tuple input) throws IOException
    {
        if (input == null || input.size() == 0)
            return null;
        try{
            String str = (String)input.get(0);
            return str.toUpperCase();
        }catch(Exception e)
        {
            throw new IOException("Caught exception processing input row ", e);
        }
    }
}
```

- Create the jar file and export it into the specific directory



- Create a text file (pigsample) in your local machine and insert the list of tuples.

```
Gaurav,24,5.6  
Rohan,22,2.5  
Ronit,25,4.5
```

- Upload the text files on HDFS in the specific directory
- Create a pig file(pscript.pig) in your local machine and write the below script

```
REGISTER upperconvert.jar;  
A = LOAD '/pigexample/pigsample' AS (name: chararray, age: int, gpa: float);  
B = FOREACH A GENERATE com.hadoop.TestUpper(name);  
DUMP B;
```

- Run the script in the terminal to get the output.
- `$pig pscript.pig`

```
2019-04-12 05:01:40,160 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2019-04-12 05:01:40,194 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2019-04-12 05:01:40,414 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2019-04-12 05:01:40,434 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(GAURAV,24,5.6)
(ROHAN,22,2.5)
(RONIT,25,4.5)
2019-04-12 05:01:41,687 [main] INFO org.apache.pig.Main - Pig script completed in 5 minutes, 57 seconds and 115 milliseconds (357115 ms)
codegyani@ubuntu64server:~$
```

Data Processing Operator

- In Pig Latin, there are several data processing operators available to manipulate and transform data.
- These operators allow you to perform various operations on your data.

LOAD: Loads data from a specified source (e.g., file, HDFS) into a relation.

```
myData = LOAD 'data.txt' USING PigStorage(',') AS (col1:chararray,  
col2:int);
```

FILTER: Selects tuples from a relation that satisfy a specified condition.

```
filteredData = FILTER myData BY col2 > 10;
```

FOREACH: Applies a transformation or computation on each tuple of a relation.

```
transformedData = FOREACH myData GENERATE col1, col2 * 2 AS  
doubledCol2;
```

GROUP BY: Groups the data based on one or more fields.

```
groupedData = GROUP myData BY col1;
```

JOIN: Combines two or more relations based on a common field.

```
joinedData = JOIN myData BY col1, otherData BY col1;
```

ORDER BY: Sorts the data based on one or more fields.

```
sortedData = ORDER myData BY col2 DESC;
```

DISTINCT: Removes duplicate tuples from a relation.

```
uniqueData = DISTINCT myData;
```

UNION: Combines two or more relations into a single relation.

```
combinedData = UNION myData, otherData;
```


SPLIT: Divides a relation into multiple relations based on a condition.

```
SPLIT myData INTO highValues IF col2 > 50, lowValues IF col2 <= 50;
```

LIMIT: Restricts the number of tuples in a relation to a specified limit.

```
limitedData = LIMIT myData 100;
```

STORE: Writes the relation to a specified output location.

```
STORE myData INTO 'output.txt' USING PigStorage(',');
```

Hive

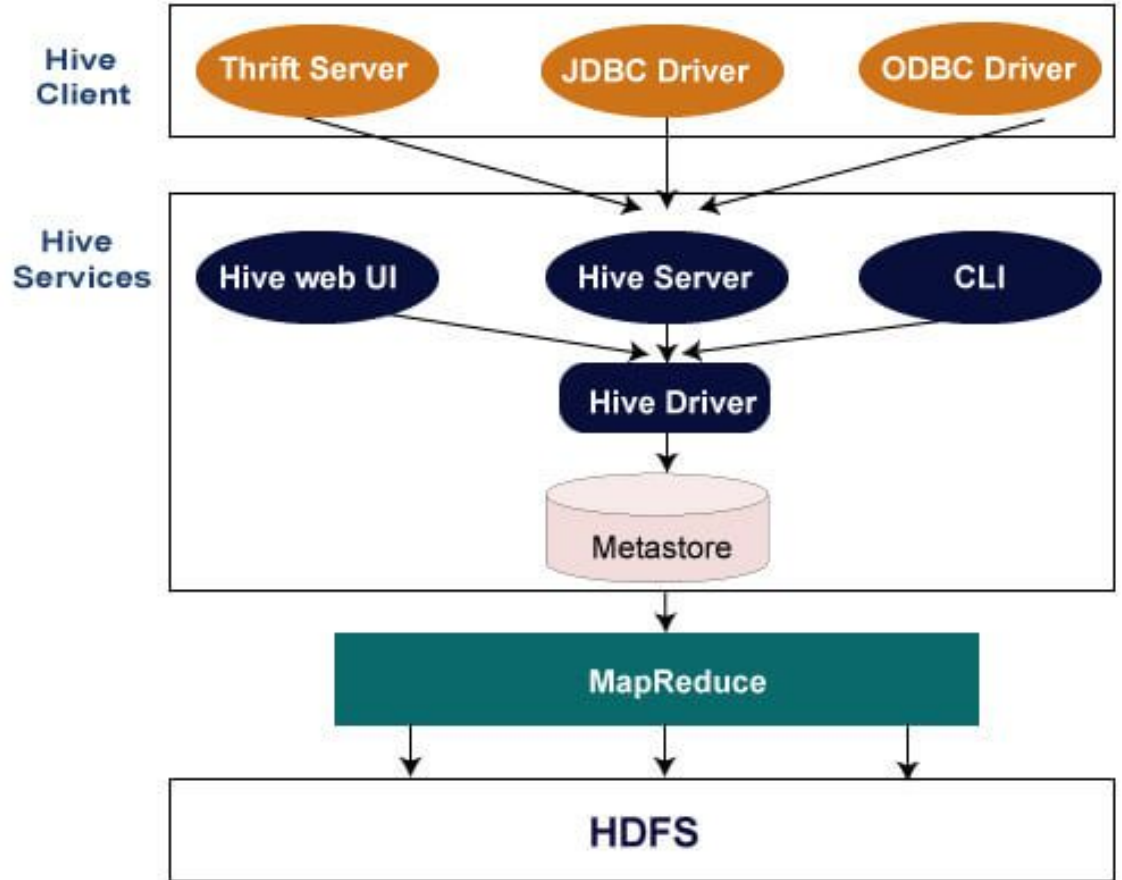
- Hive is a data warehouse system which is used to analyze structured data.
- It is built on the top of Hadoop and developed by Facebook.
- Hive provides the functionality of reading, writing, and managing large datasets residing in distributed storage.
- It runs SQL like queries called **HQL** (Hive query language) which gets internally converted to MapReduce jobs.
- Hive helps to skip the requirement of the traditional approach of writing complex MapReduce programs.
- Hive supports Data Definition Language (DDL), Data Manipulation Language (DML), and User Defined Functions (UDF).

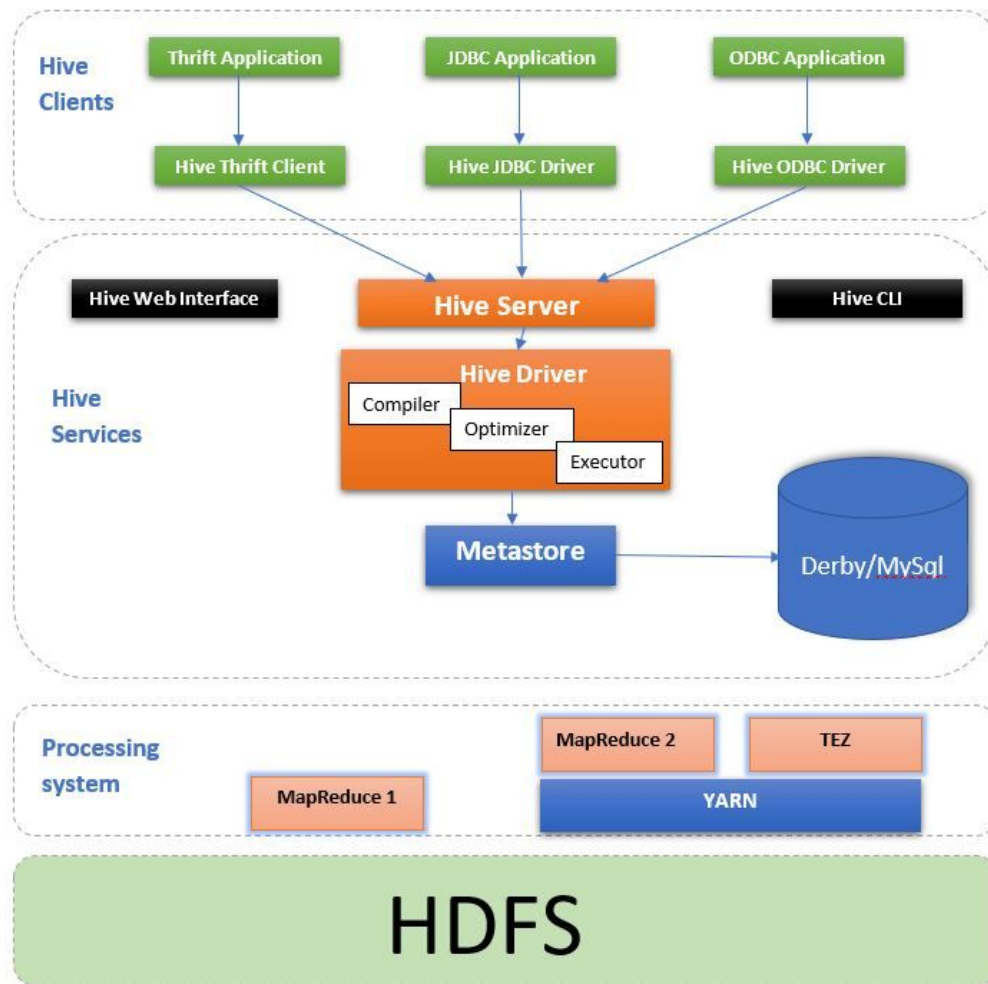
Features of Hive

- Hive is fast and scalable.
- It provides SQL-like queries that are implicitly transformed to MapReduce or Spark jobs.
- It is capable of analyzing large datasets stored in HDFS.
- It allows different storage types such as plain text, RCFile, and HBase.
- It uses indexing to accelerate queries.
- It can operate on compressed data stored in the Hadoop ecosystem.
- It supports user-defined functions (UDFs) where user can provide its functionality.

Hive Architecture

The architecture explains the flow of submission of query into Hive.





Hive Client

A component that enables clients to submit queries to Hive and retrieve results over a network connection.

It supports different types of clients such as:-

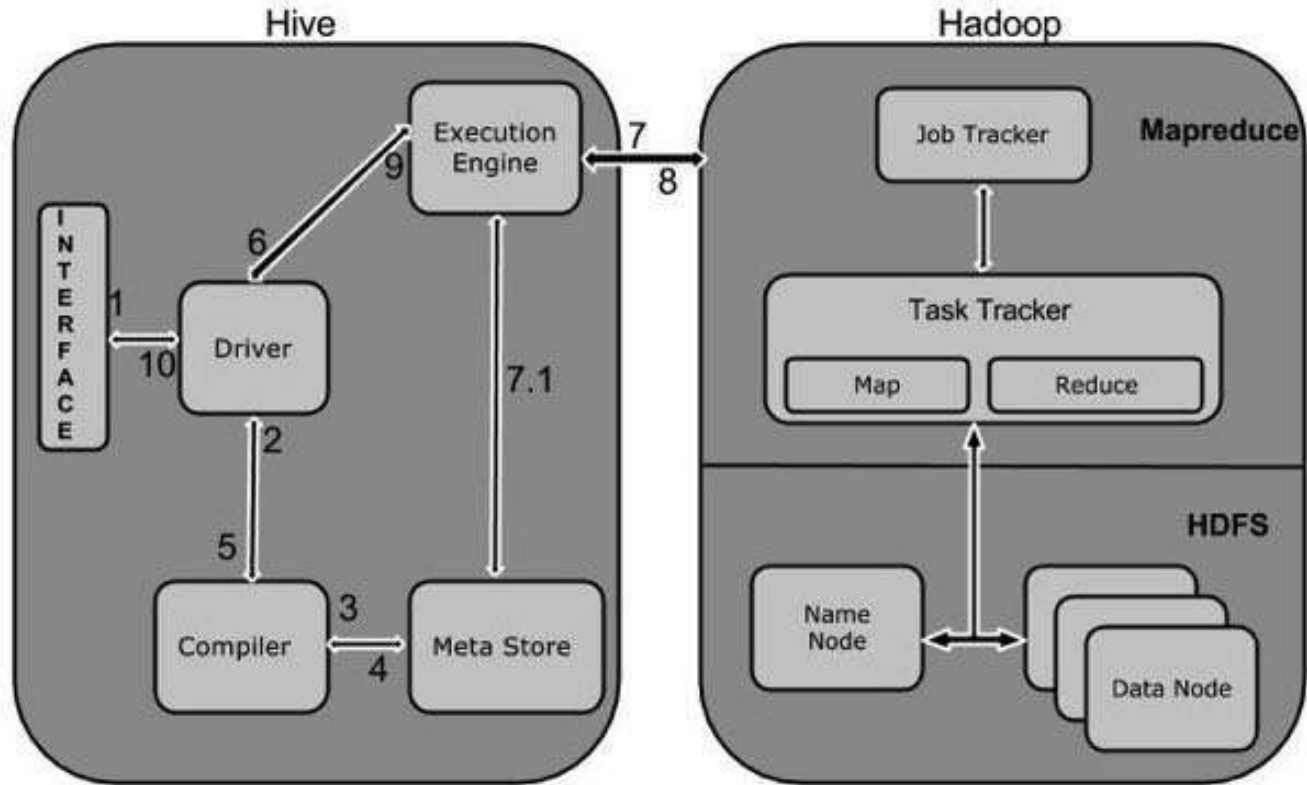
- **Thrift Server** - It is a cross-language service provider platform that serves the request from all those programming languages that supports Thrift.
- **JDBC Driver** - It is used to establish a connection between hive and Java applications. The JDBC Driver is present in the class `org.apache.hadoop.hive.jdbc.HiveDriver`.
- **ODBC Driver** - It allows the applications that support the ODBC protocol to connect to Hive.

Hive Services

- **Hive CLI** - The Hive CLI (Command Line Interface) is a shell where we can execute Hive queries and commands.
- **Hive Web User Interface** - The Hive Web UI is just an alternative of Hive CLI.
 - It provides a web-based GUI for executing Hive queries and commands.
- **Hive MetaStore** - It is a central repository that stores all the structure information of various tables and partitions in the warehouse.
 - It also includes metadata of column and its type information, the serializers and deserializers which is used to read and write data and the corresponding HDFS files where the data is stored.

- **Hive Server** - It is referred to as Apache Thrift Server.
 - It accepts the request from different clients and provides it to Hive Driver.
- **Hive Driver** - It receives queries from different sources like web UI, CLI, Thrift, and JDBC/ODBC driver.
 - It transfers the queries to the compiler.
- **Hive Compiler** - The purpose of the compiler is to parse the query and perform semantic analysis on the different query blocks and expressions.
 - It converts HiveQL statements into MapReduce jobs.
- **Hive Execution Engine** - Optimizer generates the logical plan in the form of DAG of mapreduce tasks and HDFS tasks. In the end, the execution engine executes the incoming tasks in the order of their dependencies.

Working of Hive



The following table defines how Hive interacts with Hadoop framework

Step No	Operation
1	Execute Query The Hive interface such as Command Line or Web UI sends query to Driver (any database driver such as JDBC, ODBC, etc.) to execute.
2	Get Plan The driver takes the help of query compiler that parses the query to check the syntax and query plan or the requirement of query.
3	Get Metadata The compiler sends metadata request to Metastore (any database).

Step No	Operation
4	Send Metadata Metastore sends metadata as a response to the compiler.
5	Send Plan The compiler checks the requirement and resends the plan to the driver. Up to here, the parsing and compiling of a query is complete.
6	Execute Plan The driver sends the execute plan to the execution engine.

Step No	Operation
7	Execute Job Internally, the process of execution job is a MapReduce job. The execution engine sends the job to JobTracker, which is in Name node and it assigns this job to TaskTracker, which is in Data node. Here, the query executes MapReduce job.
7.1	Metadata Ops the execution engine can execute metadata operations with Metastore.
8	Fetch Result The execution engine receives the results from Data nodes.
9	Send Results The execution engine sends those resultant values to the driver and The driver sends the results to Hive Interfaces.

Comparison with Traditional Databases

RDBMS	HIVE
It is used to maintain database	It is used to maintain data warehouse
It uses SQL (Structured Query Language).	It uses HQL (Hive Query Language).
Schema is fixed in RDBMS.	Schema varies in it.
Normalized data is stored.	Normalized and denormalized both type of data is stored.
It doesn't support partitioning.	It supports automation partition.

HiveQL

- The Hive Query Language (HiveQL) is a query language for Hive to process and analyze structured data in a Metastore.

Hive Commands

- Hive supports Data definition Language(DDL), Data Manipulation Language (DML) and User defined functions.
- Hive DDL Commands
 - create database
 - drop database
 - create table
 - drop table
 - alter table
 - create index
 - create view

- Hive DML Commands

- Select
- Where
- Group By
- Order By
- Load Data
- Join:
 - Inner Join
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join

DML Commands

Employee			Employee Department	
EMP ID	Emp Name	Address	Emp ID	Department
1	Rose	US	1	IT
2	Fred	US	2	IT
3	Jess	In	3	Eng
4	Frey	Th	4	Admin

LOAD DATA

```
hive> LOAD DATA LOCAL INPATH './usr/Desktop/kv1.txt' OVERWRITE INTO TABLE Employee;
```

SELECTS and FILTERS

```
hive> SELECT E.EMP_ID FROM Employee E WHERE E.Address='US';
```

GROUP BY

```
hive> SELECT E.EMP_ID FROM Employee E GROUP BY E.Addresss;
```


DML Commands

Employee			Employee Department	
EMP ID	Emp Name	Address	Emp ID	Department
1	Rose	US	1	IT
2	Fred	US	2	IT
3	Jess	In	3	Eng
4	Frey	Th	4	Admin

LOAD DATA

```
hive> LOAD DATA LOCAL INPATH './usr/Desktop/kv1.txt' OVERWRITE INTO TABLE Employee;
```

SELECTS and FILTERS

```
hive> SELECT E.EMP_ID FROM Employee E WHERE E.Address='US';
```

GROUP BY

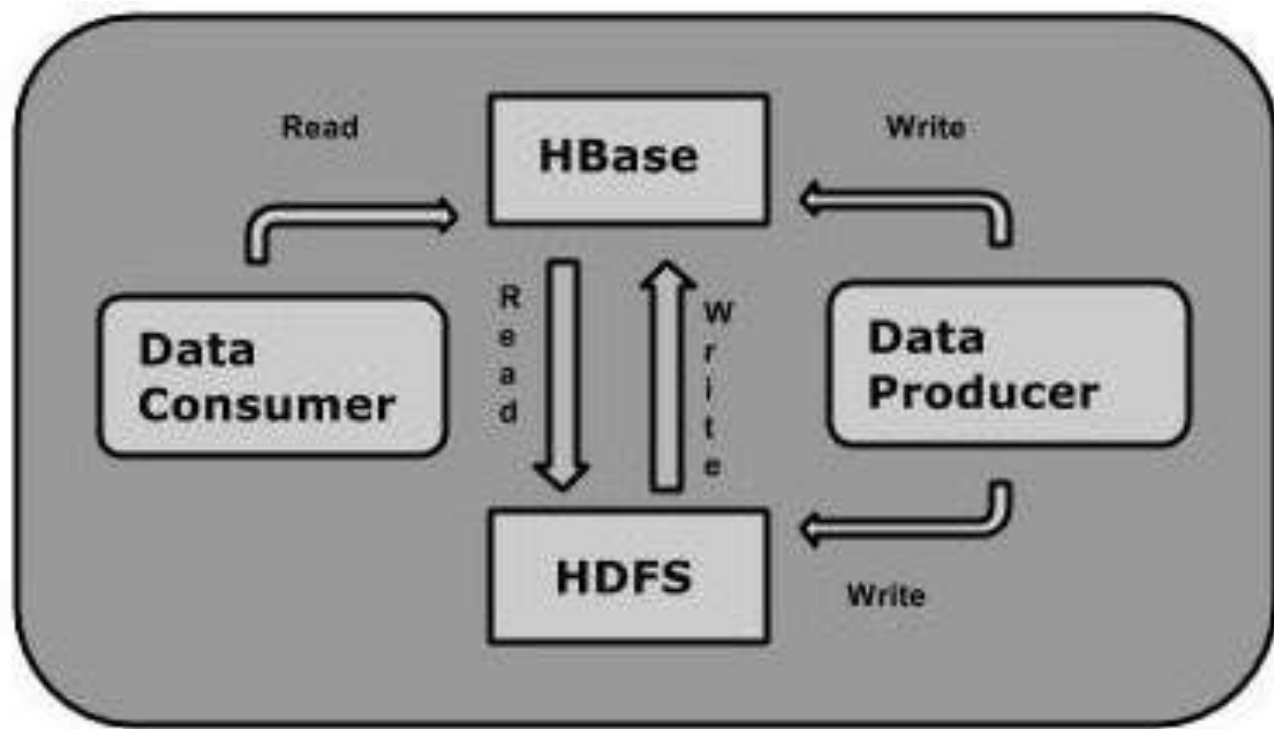
```
hive> SELECT E.EMP_ID FROM Employee E GROUP BY E.Addresss;
```

Limitations of Hadoop

- Hadoop can perform only batch processing, and data will be accessed only in a sequential manner, one has to search the entire dataset even for the simplest of jobs.
- A huge dataset when processed results in another huge data set, which should also be processed sequentially.
- At this point, a new solution is needed to access any point of data in a single unit of time (random access).
- Applications such as [HBase](#), Cassandra, couchDB, Dynamo, and MongoDB are some of the databases that store huge amounts of data and access the data in a random manner.

HBase

- HBase is a distributed **column-oriented** database built on top of the Hadoop file system.
 - **stores and retrieves data in a column-wise, the data in a table is stored and accessed column by column rather than row by row.**
 - **The values of each column are stored together in contiguous memory locations.**
- HBase is a data model that is similar to Google's big table designed to provide quick random access to huge amounts of structured data.
- It is a part of the Hadoop ecosystem that provides random real-time read/write access to data in the Hadoop File System.
- One can store the data in HDFS either directly or through HBase. Data consumer reads/accesses the data in HDFS randomly using HBase.
- HBase sits on top of the Hadoop File System and provides read and write access



HBase and HDFS

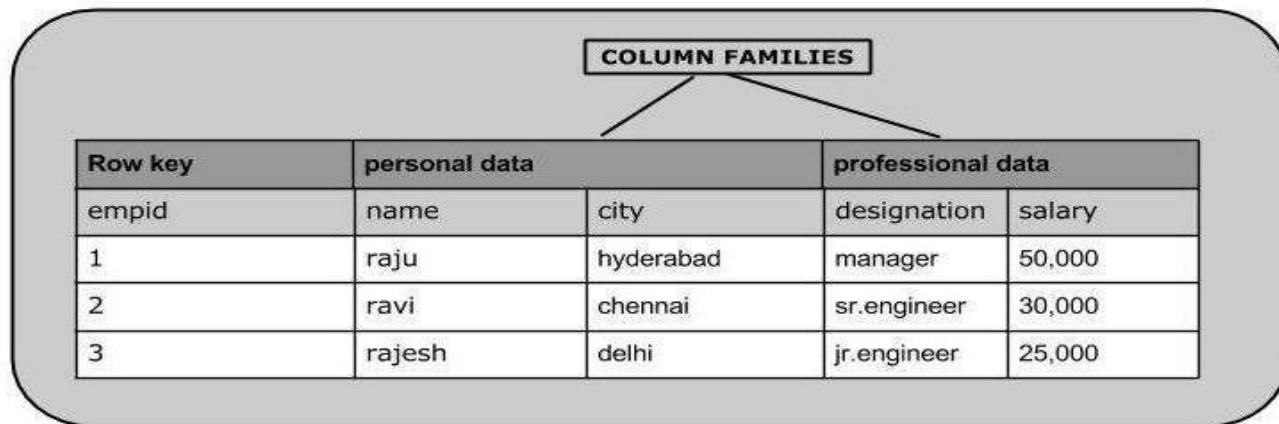
HDFS	HBASE
HDFS is a distributed file system suitable for storing large files.	HBase is a database built on top of the HDFS
HDFS does not support fast individual record lookups.	HBase provides fast lookups for larger tables
It provides only sequential access of data	HBase internally uses Hash tables and provides random access, and it stores the data in indexed HDFS files for faster lookups.

Storage Mechanism

- HBase is a column-oriented database and the tables in it are sorted by row.
- The table schema defines only column families, which are the key value pairs.
- A table have multiple column families and each column family can have any number of columns.
- Subsequent column values are stored contiguously on the disk.
- Each cell value of the table has a timestamp.
- In short, in an Hbase:
 - Table is a collection of rows.
 - Row is a collection of column families.
 - Column family is a collection of columns.
 - Column is a collection of key value pairs

Example schema of table in HBase

Rowid	Column Family			Column Family			Column Family			Column Family		
	col1	col2	col3	col1	col2	col3	col1	col2	col3	col1	col2	col3
1												
2												
3												



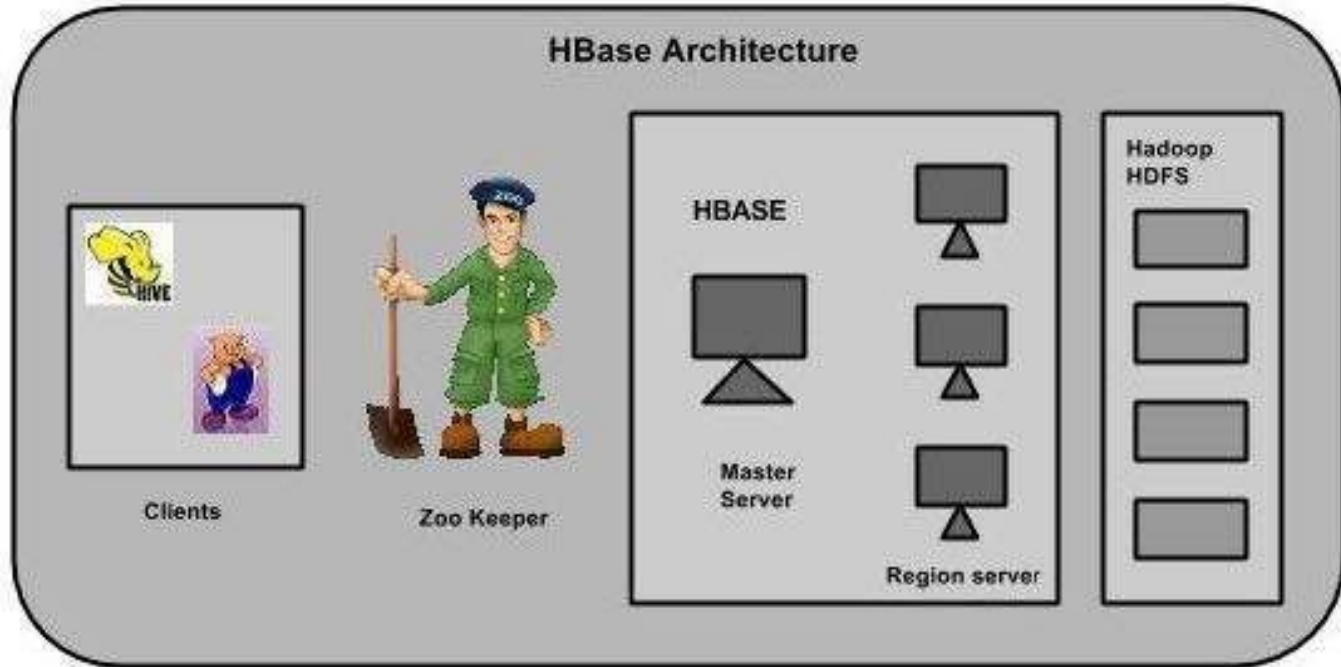
Features

- HBase is linearly scalable.
- It has automatic failure support.
- It provides consistent read and writes.
- It integrates with Hadoop, both as a source and a destination.
- It has easy java API for client.
- It provides data replication across clusters.

Where to use & Applications

- Apache HBase is used to have random, real-time read/write access to Big Data.
- It hosts very large tables on top of clusters of commodity hardware.
- Apache HBase is a non-relational database modeled after Google's Bigtable.
- It is used whenever there is a need to write heavy applications.
- Companies such as Facebook, Twitter, Yahoo, and Adobe use HBase internally.

Hbase - Architecture



- HBase has three major components: the client library, a master server, and region servers.
- Region servers can be added or removed as per requirement.

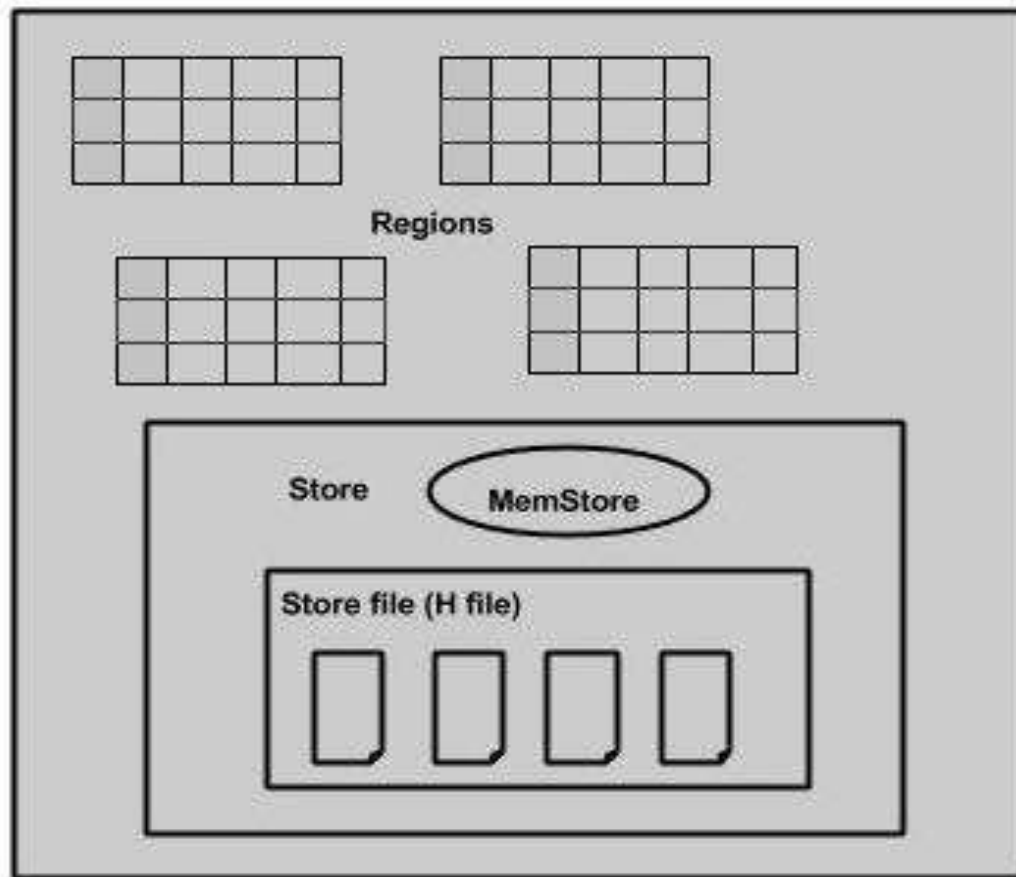
MasterServer

- Master Server Assigns regions to the region servers and takes the help of Apache ZooKeeper for this task.
- It handles load balancing of the regions across region servers.
- It unloads the busy servers and shifts the regions to less occupied servers.
- It maintains the state of the cluster by negotiating the load balancing.
- It is responsible for schema changes and other metadata operations such as creation of tables and column families.

Regions & Region Servers

- Regions are tables that are split up and spread across the region servers.
- **Region server**
 - The region servers have regions that -
 - Communicate with the client and handle data-related operations.
 - Handle read and write requests for all the regions under it.
 - Decide the size of the region by following the region size thresholds.

Region Server



Zookeeper

- Zookeeper is an open-source project that provides services like maintaining configuration information, naming, providing distributed synchronization, etc.
- Zookeeper has ephemeral nodes representing different region servers. Master servers use these nodes to discover available servers.
- In addition to availability, the nodes are also used to track server failures or network partitions.
- Clients communicate with region servers via zookeeper.
- In pseudo and standalone modes, HBase itself will take care of zookeeper.